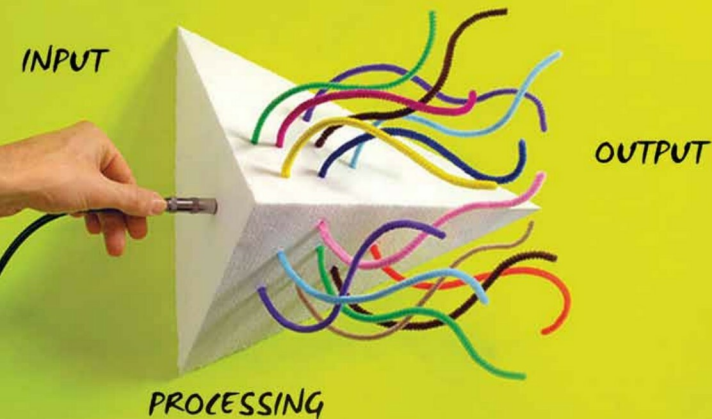




I/O Control in Linux

This article, which is part of the series on Linux device drivers, talks about the typical `ioctl()` implementation and usage in Linux.

“Get me a laptop, and tell me about the x86 hardware interfacing experiments in yesterday’s Linux device drivers’ lab session, and also about what’s planned for the next session,” cried Shweta, exasperated at being confined to bed due to food poisoning at a friend’s party.

Shweta’s friends summarised the session, and told her that they didn’t know what the upcoming sessions, though related to hardware, would be about. When the doctor requested them to wait outside, they took the opportunity to plan and talk about the most common hardware-controlling operation, `ioctl()`.

Introducing `ioctl()`

Input/Output Control (`ioctl`, in short) is a common operation, or system call, available in most driver categories. It is a

‘one bill fits all’ kind of system call. If there is no other system call that meets a particular requirement, then `ioctl()` is the one to use. Practical examples include volume control for an audio device, display configuration for a video device, reading device registers, and so on—basically, anything to do with device input/output, or device-specific operations, yet versatile enough for any kind of operation (for example, for debugging a driver by querying driver data structures). The question is: how can all this be achieved by a single function prototype? The trick lies in using its two key parameters: *command* and *argument*. The *command* is a number representing an operation. The *argument* is the corresponding parameter for the operation. The `ioctl()` function implementation does a ‘switch ... case’ over the *command* to implement the corresponding functionality. The following has been its prototype in the Linux kernel for quite some time: